

Day 4 — RAG, embeddings, advanced retrieval, and evaluation

Outcome

Be able to design and debug a secure, fresh, measurable RAG system from source ingestion to grounded answer, while separating retrieval failures from generation failures.

1. Core mental model

Retrieval-Augmented Generation (RAG) is an open-book workflow for an LLM:

```
question
→ retrieve trusted evidence
→ build focused context
→ generate from evidence
→ validate and cite
```

It addresses private/current knowledge, large corpora, citations, and model knowledge limitations. It does not automatically make the model correct.

```
answer quality
≈ retrieval quality
× context quality
× generation quality
```

2. Offline ingestion flow

2.1 Collect and identify sources

Sources can be PDFs, web/wiki pages, documents, database records, tickets, product data, or API content.

Assign:

- tenant and document IDs;
- authoritative source/owner;
- source URI;
- version/status;
- effective/expiry dates;
- access scope;
- checksum and timestamps.

2.2 Parse

Extract text plus structure:

- headings;
- lists;
- tables;
- pages;
- links;

- section boundaries.

Parsing quality sets an upper bound on retrieval. Broken tables, repeated headers, OCR errors, or joined columns create bad chunks.

2.3 Clean, normalize, and deduplicate

Remove repeated headers/footers, navigation, broken symbols, duplicate paragraphs, and irrelevant boilerplate. Normalize encoding, dates, titles, and business labels.

Use content hashes and canonical source/version rules to avoid duplicate indexing.

2.4 Chunk

Chunk by source structure where possible:

- sections for policies;
- paragraphs for articles;
- rows/table-aware groups for tables;
- functions/classes for code;
- turns for conversations.

Size trade-off

Small chunks are focused but lose surrounding meaning. Large chunks preserve context but add noise, token cost, and less precise representations.

Overlap protects boundaries but excessive overlap creates duplicate retrieval and artificial confidence.

Advanced strategies:

- structure-aware/heading-based;
- semantic/adaptive;
- parent-child;
- hierarchical;
- summary indexing;
- table-aware.

2.5 Embed

chunk text → embedding model → vector

Use compatible embedding spaces for indexing and queries. Store the embedding model/version with the index. Do not mix incompatible vectors.

Embedding selection considers domain language, required languages, input limits, vector dimension, latency, cost, privacy, deployment, and migration.

Similarity and index choices

- Cosine similarity compares vector direction.
- Dot product measures alignment and magnitude; for normalized vectors it closely relates to cosine similarity.
- Euclidean distance measures geometric distance.

Use the metric expected by the embedding model and index configuration.

Vector index families:

Index	Behavior	Trade-off
Flat	Exact comparison with all vectors	Highest recall and useful baseline; slow at scale.
HNSW	Graph-based approximate nearest-neighbor search	Strong recall/latency, fast queries, higher memory.
IVF	Cluster vectors and search selected partitions	Efficient at scale; needs clustering and recall depends on partitions searched.

Approximate nearest-neighbor search exchanges some recall for lower latency. Establish a Flat baseline when practical and tune against evaluation data.

2.6 Store and publish

Vector record:

```
{
  "tenant_id": "tenant-17",
  "document_id": "policy-123",
  "chunk_id": "policy-123-07",
  "text": "Regional campaigns require compliance approval.",
  "embedding_version": "embed-v3",
  "document_version": "4.2",
  "status": "approved",
  "region": "India",
  "allowed_groups": ["marketing", "legal"],
  "page": 12,
  "section": "Regional approval"
}
```

Track ingestion state, parser/chunker versions, chunk count, failures, skipped pages, and index publication status.

An ingestion pipeline should be incremental, idempotent, deletion-aware, versioned, retryable, recoverable, observable, and ACL-aware.

3. Online query flow

```
authenticate
→ derive tenant/user/groups
→ validate and rate-limit
→ normalize/rewrite query
→ apply mandatory security/lifecycle filters
→ retrieve candidates
→ fuse/deduplicate/rerank
→ assemble token-bounded context
→ grounded generation
→ schema/citation/safety validation
→ respond/stream
→ trace and collect feedback
```

Security before retrieval

Apply tenant, role/group, region, classification, document status, and validity filters before unauthorized content can enter model context.

Do not retrieve broadly and ask the model to hide forbidden content.

Candidate retrieval

Dense/vector search:

- semantic similarity;
- paraphrases;
- natural questions.

Sparse/keyword search:

- IDs;
- names;
- acronyms;
- error codes;
- legal phrases;
- exact versions.

BM25 is a common lexical ranking method based on term frequency and rarity. It is strong for exact identifiers and rare domain terms but can miss semantic paraphrases, which is why it is often paired with dense retrieval.

Hybrid retrieval combines both, merges/fuses results, removes duplicates, and reranks.

Reciprocal Rank Fusion (RRF) combines ranked lists without assuming their raw scores are comparable:

```
fusion_score(chunk)
= 1 / (constant + vector_rank)
+ 1 / (constant + keyword_rank)
```

A chunk that ranks highly in either or both paths receives a stronger fused score. The fusion constant and candidate counts are tuning choices; evaluate them rather than treating source-note example values as universal defaults.

Top-k

Low top-k:

- lower latency/cost/noise;
- may miss necessary evidence.

High top-k:

- better candidate recall;
- more irrelevant content and reranking cost.

A common architecture retrieves broadly, reranks, then sends a small best set. Tune all values against an evaluation dataset rather than copying universal numbers.

Reranking

First-stage retrieval is fast candidate generation. A cross-encoder, small LLM, fusion method, business scoring, or metadata/freshness rule performs closer question-chunk comparison.

Keep reranking distinct from contextual compression:

- reranker selects/reorders evidence;
- compression extracts only relevant parts from selected evidence.

Query rewriting and multi-query

Rewriting bridges user and document vocabulary. Multi-query generates several search formulations for ambiguous or broad needs.

Risks:

- meaning drift;
- increased calls, cost, and duplicate results.

Log original and rewritten queries and evaluate whether rewriting helps.

Context assembly

- enforce token budget;
- remove duplicates;
- preserve headings and citations;
- prefer approved/current sources;
- group compatible evidence;
- do not combine conflicting versions silently;
- keep instructions separate from untrusted retrieved text.

Context pollution means irrelevant, duplicate, stale, contradictory, or badly parsed chunks enter the prompt. More context can reduce answer quality while increasing latency, tokens, and injection surface.

Grounded prompt

Use only the supplied evidence.
Do not invent missing policy.
If evidence is missing or contradictory, say so.
Cite each major claim using the supplied source identifiers.

Low temperature can increase consistency but cannot guarantee factuality.

Do not use the LLM's self-reported confidence as the confidence score. Derive a risk/answerability signal from retrieval relevance, citation coverage, source authority/freshness, reranker scores, contradiction checks, groundedness, and required metadata. Weak evidence, conflicting versions, or missing user context should lead to clarification, abstention, or escalation.

4. Storage roles

- Object storage: original documents and parsed artifacts.
- Relational database: catalog, ownership, ingestion state, interactions, feedback, audit.

- Vector store: chunk vectors plus retrieval metadata.
- Keyword/search index: lexical retrieval.
- Cache: carefully scoped stable parsing, embeddings, retrieval, reranking, or responses.

Cache keys may need tenant, access-scope hash, query, index version, embedding/reranker/prompt/model versions. Never cross permission scopes.

Vector-tool positioning

Tool	Source-note positioning
FAISS	Local/custom vector-search library; persistence, metadata, replication, and APIs remain your responsibility.
Chroma	Developer-friendly vector store for prototypes and smaller applications.
Qdrant	Purpose-built vector database with filtering and self-hosted/managed options.
Pinecone	Managed vector database emphasizing operational convenience.
Weaviate	Open-source vector database with semantic and hybrid-search capabilities.
Milvus	Open-source vector database positioned for large-scale vector search and infrastructure control.

Select from scale, metadata filtering, tenant isolation, availability, hybrid requirements, operational expertise, deployment constraints, and cost—not from name alone.

5. Evaluation model

Retrieval metrics

- Recall@k: required evidence retrieved?
- Precision@k/context precision: how much retrieved context is relevant?
- MRR: how early is the first relevant result?
- nDCG: how good is the overall graded ordering?
- Context recall: were all pieces of evidence required for the answer retrieved?
- Empty retrieval rate.
- Expected document/section hit.

Context precision asks whether the supplied chunks are mostly useful. Context recall asks whether the retriever found every necessary fact or condition. A response can retrieve one correct section early and still have weak context recall if another required exception is missing.

Generation metrics

- Groundedness: is each material claim supported by the supplied context?
- Faithfulness: does the answer accurately represent that context without contradiction or distortion?

- Answer relevance: does it address the question?
- Correctness/required facts.
- Citation correctness/coverage.
- Abstention/refusal correctness.
- Safety/toxicity where applicable.

Groundedness and faithfulness are closely related and some evaluators use the labels differently. Define the rubric and evaluator contract rather than comparing scores with incompatible meanings.

System/business metrics

- retrieval/reranker/model/end-to-end latency;
- tokens and cost;
- errors, fallbacks, cache hits;
- user feedback and citation clicks;
- task resolution/escalation.

Golden dataset

Each reviewed case can include:

```
{
  "question": "Who approves reopening after shutdown?",
  "expected_document_ids": ["OPS-482"],
  "expected_sections": ["7.3"],
  "required_facts": ["engineering inspection", "manager approval"],
  "forbidden_documents": ["OPS-482-old"],
  "expected_abstention": false
}
```

Include happy, difficult, no-answer, adversarial, multilingual, conflicting-document, permission-boundary, and stale/deleted-source cases.

Run evaluation after changes to parser, chunks, embeddings, index, filters, top-k, retrieval, reranker, prompt, model, or metadata.

Offline evaluation supports release decisions. Online feedback supplies production cases. Feedback must retain the question, retrieved chunks, component versions, output, and user signal.

Human evaluation remains important for nuanced correctness, completeness, policy interpretation, high-risk answers, and rubric calibration. Reviewers need the retrieved context and citations, not only the final prose.

Continuous improvement is a controlled loop:

```
collect representative questions
→ classify the failure
→ improve parsing/chunking/retrieval/prompt/fallback
→ evaluate offline against the baseline
→ deploy safely
→ monitor online feedback and escalations
→ repeat
```

Do not automatically train on every user correction; validate authority, privacy, and quality first.

Retrieval drift

Retrieval drift means search quality degrades over time even when the service remains technically healthy. Causes in the source notes include new document formats, changed terminology or question patterns, collection growth, duplicate content, metadata changes, and embedding-model changes.

Monitor Recall@k, Precision@k, MRR, no-result rate, reranker-score distribution, user reformulation, and citation interaction over time. Diagnose the changed stage before retuning the LLM.

6. Failure diagnosis

Correct document was never indexed

Inspect connector cursor, parser status, ingestion job, deletion/version rules, and index publication.

Correct evidence exists but was not retrieved

Inspect chunks, embedding fit, keyword path, filters, query rewriting, top-k, and index version.

Evidence retrieved but answer is wrong

Inspect context noise/order, conflicting sources, prompt, answerability, structured output, and model behavior.

Unauthorized evidence retrieved

Treat as a security incident: access filter, tenant scope, cache key, ACL propagation, or index isolation failed.

Stale/deleted content appears

Inspect version/status/effective date, deletion tombstones, connector reconciliation, index publication, and cache invalidation.

High cost or latency

Measure each stage before changing quality levers. Remove unnecessary LLM calls, duplicate context, broad tools, and repeated embeddings; improve filters; use model tiers; cache safe stable work; run independent retrieval paths concurrently.

7. Production risks and controls

Risk	Control
Poor parsing	Difficult-document benchmark and parse-quality checks.
Bad chunks	Source-aware chunking and golden retrieval tests.
Missing metadata	Planned identity/lifecycle/security/citation schema.

Duplicate documents	Hashes, source/version IDs, canonical ownership.
Low recall	Better chunks/embeddings, hybrid search, rewrite, broader candidates.
Low precision	Filters, thresholds, dedupe, reranking, smaller final context.
Hallucination	Grounding, answerability, citations, claim checks, abstention.
Prompt injection in documents	Treat context as untrusted, isolate instructions, restrict tools.
Cross-tenant leakage	Server-derived tenant, mandatory ACL filters, scoped cache/index, tests.
Stale knowledge	Incremental/versioned ingestion, active status, freshness SLO, deletion reconciliation.
Provider outage	Timeout, fallback or controlled failure; preserve evidence trace.
Reranker outage	Graceful degradation to first-stage ranking when risk permits.

8. Vanilla versus advanced RAG

Vanilla RAG is enough when:

- corpus and access model are simple;
- questions are straightforward;
- vector retrieval has acceptable quality;
- one retrieve-and-generate path works;
- latency/cost constraints favor simplicity.

Advanced RAG is justified by measurable failures:

- exact identifiers missed;
- long or structured documents;
- ambiguous conversational queries;
- multi-document comparison;
- noisy candidates;
- strict citations/answerability;
- freshness and authorization complexity.

Start simple, establish a baseline, then add only mechanisms that improve measured cases.

9. Enterprise policy-assistant example

Offline:

```
policy sources
→ incremental connector
→ parse tables/headings
→ normalize and deduplicate
→ attach version/effective date/ACL
→ parent-child chunks
→ embeddings + keyword index
→ evaluated index publication
```

Online:

```
employee identity/groups
→ tenant/ACL filter
→ query rewrite
→ dense + keyword retrieval
→ fusion + reranking
→ current approved evidence
→ grounded answer with citations
→ validation, telemetry, feedback
```

If policies conflict, return the conflict and escalate rather than inventing authority.

Project-grounded example: benchmark-aware RAG over a deterministic platform

Project scenario. The first **DPDK Automation for Network Packet Processing** project produced the truth-bearing assets: AMD-centric tuning documents, benchmark templates, raw logs, parsed metrics, run metadata, and comparisons across BIOS, OS, compiler, CPU SKU, and benchmark variations. **DPDK BenchOps Copilot** then made those fragmented assets searchable and usable through a LlamaIndex ingestion pipeline and a grounded query workflow.

How the ingestion concepts apply.

```
benchmark documents + logs + database JSON + run metadata + internal notes
→ normalize source formats
→ create phase-aware chunks for setup, execution, metrics, and
interpretation
→ attach benchmark/platform/source/run/provenance metadata
→ store authoritative records and artifacts in Postgres and S3/MinIO
→ store embeddings in a vector database
```

The phase-aware chunking decision addressed a real domain problem: a generic fixed chunk could mix setup instructions with metric interpretation or separate a performance value from its run context. Metadata such as benchmark type, AMD server generation, source type, run context, and provenance enabled context-aware filtering rather than relying on semantic similarity alone.

How the online concepts apply.

engineer asks a tuning or regression question

- identify intent
- retrieve benchmark-aware evidence with metadata filters
- combine hybrid retrieval with deterministic run/log access where needed
- compare runs through a deterministic tool
- verify support
- answer with citations

This separates three failure classes: missing/wrong retrieved evidence, incorrect deterministic run data, and unsupported generation. It also gives the interviewer a concrete reason to measure context precision/recall separately from groundedness and citation coverage.

Design decisions and trade-offs.

- **Truth store versus semantic index:** Postgres and S3/MinIO remained authoritative while the vector database supported discovery. This added synchronization and versioning work, but avoided making approximate retrieval the record of truth.
- **Domain-aware chunks versus naive chunks:** phase-aware chunks and rich metadata required more ingestion logic, but preserved workload and platform context needed for safe advice.
- **Hybrid retrieval and tools versus vector-only RAG:** additional stages increased latency and observability needs, but exact run data and command-related questions could not safely depend on similarity alone.
- **Verification and citations versus fastest response:** verification added work to the critical path, but the domain explicitly rejected hallucinated tuning guidance.

Outcome. The documented result was grounded benchmark assistance with contextual evidence and citations, faster analysis, and less dependence on tribal knowledge. The project provides no numeric retrieval scores, latency reduction, or productivity percentage.

Senior/Staff interview framing.

- **Senior:** take one question such as “why did these runs differ?” and trace source ingestion, metadata filtering, retrieval, `RunDiff`, context construction, answer verification, and citations. Explain how you would diagnose a missing expected run or guide section.
- **Staff:** begin with the trust constraint, then show how the earlier deterministic platform made the later RAG system possible. Discuss ownership of authoritative data, index publication, versioning, evaluation gates, failure isolation, and which measured failure would justify a more advanced retrieval stage.

Evidence boundary. The project states that retrieval was hybrid, but it does not name a lexical engine, vector database product, embedding model, similarity index family, reranker, or exact top-k values. Do not supply those details in an interview unless you can support them from another real source.

10. Interview questions

1. Why is RAG an “open-book” pattern rather than a model?
2. Why is parsing quality part of retrieval quality?
3. How do chunk size and overlap trade context against precision/cost?

4. Why must indexing/query embeddings be compatible?
5. Vector database versus retriever?
6. Keyword versus vector versus hybrid retrieval?
7. What does a reranker do, and how is it different from compression?
8. How do you choose top-k?
9. Recall@k versus precision@k versus groundedness?
- 10
 - . Why must authorization occur before prompt construction?
- 11
 - . How do you propagate document updates, ACL changes, and deletions?
- 12
 - . How do you debug "the answer is wrong" without immediately changing the LLM?
- 13
 - . When is Vanilla RAG sufficient?
- 14
 - . RAG versus fine-tuning?
- 15
 - . What is BM25 good at, and why combine it with dense retrieval?
- 16
 - . How is retrieval drift different from a vector-store outage?

11. Exit checklist

- [] Draw ingestion and query flows from memory.
- [] Design chunk and metadata schemas.
- [] Explain embeddings, vector stores, keyword search, hybrid, reranking, and compression.
- [] Build a golden evaluation case and choose metrics.
- [] Diagnose indexing, retrieval, generation, authorization, and freshness failures separately.
- [] Design safe caching and versioning.
- [] Explain when each advanced RAG technique is justified.
- [] Distinguish groundedness, faithfulness, answer relevance, and retrieval drift.

Source notes

- [NLP Fundamentals](#)
- [LLM Fundamentals](#)
- [Databases for AI Systems](#)
- [RAG for Enterprise Knowledge](#)
- [Embeddings and Vector Databases](#)
- [Advanced RAG Patterns](#)
- [Evaluating RAG Systems](#)
- [Enterprise GenAI Solution Design](#)
- [Vanilla RAG End to End](#)
- [LlamaIndex End to End](#)
- [Capstone Revision Day 2](#)
- [DPDK Automation for Network Packet Processing](#)

- DPDK BenchOps Copilot